Multi-Objective Portfolio Optimisation Using Metaheuristic Algorithms

A Comparative Study of NSGA-II, MOEA/D, AGE-MOEA and Exact MILP on a 100-Asset Universe

Shetty Arya Dinesh (1009346)
Buraporn Ruangchaem (1009099)
Clarence Elvareta Kurniawan (1009166)

40.018 Heuristics and Systems Theory
Singapore University of Technology and Design

Team Member	**Contribution**
Clarence Elvareta Kurniawan (1009166)	Introduction, Problem Formulation
Shetty Arya Dinesh (1009346)	Methodology
Buraporn Ruangchaem (1009099)	Results, Discussion

Abstract

This report addresses a joint asset-selection-and-allocation problem: choosing at most $K = 15$ assets from a heterogeneous $M = 100$-asset universe (equities, bonds, broad-market ETFs and cryptocurrency) to trace the Pareto-optimal frontier between annualised expected return and Conditional Value-at-Risk ($\text{CVaR}_{0.95}$). Because the cardinality constraint introduces M binary decision variables, the problem is a Mixed-Integer Linear Program (MILP) with a search space of $\binom{100}{15} \approx 2.5 \times 10^{14}$, which is NP-hard and computationally challenging as M grows. We benchmark exact ε-constraint MILP optimisation (Gurobi) against three metaheuristics: NSGA-II, MOEA/D, and AGE-MOEA; and separately study the effect of constraint-handling strategy (repair, penalty, decoder) and genetic operator choice (SBX, uniform crossover, single-point crossover) on solution quality and computational cost. We further stress-test the selected portfolio against the COVID-19 (Feb–Apr 2020) and 2025 tariff-shock windows, and statistically validate algorithm rankings over 15 independent runs using the Mann–Whitney U test.

Contents

1 Introduction **4**
1.1 Problem Motivation . . . 4
1.2 Objectives of This Study . . . 4
1.3 Cardinality Justification . . . 4

2 Problem Formulation **4**
2.1 Data . . . 4
2.2 Decision Variables . . . 5
2.3 Objectives (bi-objective Pareto front) . . . 5
2.4 Constraints . . . 5

3 Methodology **6**
3.1 Data Processing and Risk Estimation . . . 6
3.2 Codebase and Reproducibility . . . 6
3.3 Optimisation Pipeline . . . 7
3.4 Exact Solver: Gurobi ϵ-Constraint Method . . . 8
3.5 Metaheuristic Approaches . . . 8
3.6 Solution Encoding and Hybrid Architecture . . . 9
3.7 Constraint-Handling Strategies . . . 10
3.7.1 Repair . . . 10
3.7.2 Penalty . . . 11
3.7.3 Decoder . . . 11
3.8 Search Operators . . . 11
3.9 Evaluation Metrics and Experimental Tests . . . 12

4 Results **13**
4.1 Data Summary and Baseline . . . 13
4.2 Phase 1: Exact Optimisation and Gurobi Scaling . . . 13
4.2.1 Exact Pareto Front . . . 13
4.2.2 Scaling Behaviour . . . 14
4.3 Phase 2: Metaheuristic Configuration Engineering . . . 15
4.3.1 Constraint Handling . . . 15
4.3.2 Search Operators . . . 16
4.4 Phase 3: Final Algorithm Comparison . . . 16
4.4.1 Pareto Front Quality . . . 16
4.4.2 Statistical Robustness . . . 17
4.5 Phase 4: Financial Validation and Stress Testing . . . 18

5 Discussion **19**
5.1 Constraint Handling is the Primary Design Lever . . . 19
5.1.1 Why Repair Wins on Coverage but Decoder Wins on Proximity . . . 19
5.1.2 Why Penalty Fails . . . 20
5.2 Search Operators: Fine-Tuning, Not Architecture . . . 20
5.3 Algorithm Comparison . . . 20
5.3.1 NSGA-II vs. AGE-MOEA: Equivalent Quality, Different Variance . . . 20
5.3.2 MOEA/D: Fast but Structurally Mismatched . . . 20
5.4 Insights from the Selected Portfolio . . . 20
5.5 Limitations . . . 21

6 Conclusion **21**

A Appendix **24**
A.1 Codebase and Reproducibility Details 24
A.1.1 Reproduction Steps 24
A.1.2 Output Fields Used in the Report 24
A.1.3 Implementation Notes 25
A.2 Extended Stress-Testing Results 25

1 Introduction

1.1 Problem Motivation

Portfolio optimisation seeks an allocation that simultaneously maximises expected return and minimises downside risk. The classical Markowitz mean–variance formulation [1] is solvable in polynomial time when weights are continuous. Introducing a *cardinality constraint*, restricting the portfolio to at most K of M candidate assets, requires binary selection variables $z_i \in \{0, 1\}$; converting the problem into a Mixed-Integer Linear Program (MILP). This is NP-hard: no known algorithm solves it exactly in polynomial time as M grows, and exhaustive enumeration of $\binom{M}{K}$ subsets becomes computationally infeasible well before $M = 100$.

A second motivation is cross-asset diversification. Portfolios concentrated in a single sector suffer correlated drawdowns during systemic shocks. Combining equities across sectors with fixed income, broad-market ETFs, and cryptocurrency is intended to reduce tail risk without proportionally sacrificing return; this motivates the heterogeneous $M = 100$ universe used throughout this report.

1.2 Objectives of This Study

This report has four goals:

1. Evaluate the scaling behaviour of the exact ϵ-constraint Gurobi solver as M increases, and use the resulting exact frontier as a benchmark for metaheuristic performance.
2. Compare three constraint-handling strategies (repair, penalty, decoding) under an otherwise fixed metaheuristic (NSGA-II) to identify the most effective way of enforcing cardinality, linkage, sector, crypto, and bond-floor constraints.
3. Compare three genetic search operators (SBX, uniform crossover, single-point crossover) under the Decoder representation to isolate crossover effects on a fixed continuous priority-vector encoding.
4. Compare three population-based metaheuristics: NSGA-II, MOEA/D, and AGE-MOEA; against each other and against the Gurobi exact front, using Pareto-front quality indicators (HV, GD, IGD), statistical significance testing across 15 independent runs, and out-of-sample stress performance.

1.3 Cardinality Justification

Restricting the portfolio to at most $K = 15$ holdings (out of $M = 100$) is grounded in the empirical finance literature: Evans and Archer [2] and Statman [3] both show that 10–20 randomly selected assets capture the large majority of attainable diversification benefit, with marginal risk reduction flattening sharply beyond this range. Practically, transaction costs and monitoring overhead scale with the number of holdings, and active management mandates commonly cap position counts for exactly this reason.

2 Problem Formulation

2.1 Data

Daily adjusted-close prices for all $M = 100$ assets were collected via `yfinance`. The final cached panel spans **2020-04-10 to 2025-05-19** (1,866 trading days).

Note: the panel's start date is later than the originally proposed 2015 start because the four cryptocurrency tickers — most restrictively SOL-USD, which began trading in 2020 — force the intersection of all 100 assets' available history to begin only once every asset has data. Consequently the originally planned 2008–2009 Global Financial Crisis (GFC) stress window is not usable against the full 100-asset panel and is excluded from this report; COVID-19 (2020) and the 2025 tariff shock are used instead, both of which fall inside the available data range. This deviates from the original

project proposal, which planned for GFC stress testing, but is necessitated by the cryptocurrency data availability constraint.
Daily log returns $r_t = \ln(P_t/P_{t-1})$ are computed per asset; annualised expected return μ and covariance Σ use a 252-trading-day annualisation factor. CVaR is computed empirically (historical simulation) rather than assumed parametric.

2.2 Decision Variables

Let $\mathbf{w} = [w_1, \ldots, w_M]^\top$ be continuous portfolio weights and $\mathbf{z} = [z_1, \ldots, z_M]^\top \in \{0,1\}^M$ binary selection indicators, $M = 100$.

2.3 Objectives (bi-objective Pareto front)

$$\text{Maximise} \quad f_1(\mathbf{w}) = \boldsymbol{\mu}^\top \mathbf{w} \qquad \text{(annualised expected return)} \tag{1}$$

$$\text{Minimise} \quad f_2(\mathbf{w}) = \text{CVaR}_{0.95}(\mathbf{w}) \qquad \text{(expected loss, worst 5\% of scenarios)} \tag{2}$$

2.4 Constraints

$$\sum_i w_i = 1 \qquad \text{(budget: fully invested)} \tag{3}$$

$$\sum_i z_i \leq K = 15 \qquad \text{(cardinality)} \tag{4}$$

$$0.02\, z_i \leq w_i \leq 0.30\, z_i \quad \forall i \qquad \text{(linkage: weight only if selected)} \tag{5}$$

$$\sum_{i \in \mathcal{S}_k} w_i \leq 0.40 \quad \forall k \qquad \text{(sector cap)} \tag{6}$$

$$\sum_{i \in \mathcal{C}} w_i \leq 0.20 \qquad \text{(crypto cap)} \tag{7}$$

$$\sum_{i \in \mathcal{B}} w_i \geq 0.10 \qquad \text{(bond floor)} \tag{8}$$

$$z_i \in \{0,1\} \qquad \text{(integrality — source of NP-hardness)} \tag{9}$$

Cardinality is enforced as $\leq K$, not $= K$, consistent with the project proposal. This allows the optimiser to freely choose fewer than 15 holdings where diversification's marginal benefit no longer justifies additional positions. The sector cap encodes diversification directly into the feasible region. Table 1 summarises the asset universe.

Table 1: Asset universe composition ($M = 100$).

Asset class	**Count**	**Role**	**Examples**
S&P 500 equities	80	Core return driver	AAPL, NVDA, JPM, XOM, ...
Bond ETFs	6	Defensive hedge	AGG, TLT, BND, LQD, HYG, SHY
Broad/thematic ETFs	10	Factor diversification	SPY, QQQ, GLD, EEM, ...
Cryptocurrencies	4	Alternative asset	BTC-USD, ETH-USD, BNB-USD, SOL-USD
Total	**100**		

3 Methodology

The computational workflow follows the project proposal closely: build a screened multi-asset universe, estimate return and tail-risk quantities from daily returns, solve the constrained selection-and-allocation problem, and evaluate both Pareto-front quality and financial robustness. The key methodological point is that this is not only a continuous allocation problem. Each solution must first choose a small subset of assets and then assign feasible weights to the chosen assets. The binary selection decision is what makes the problem combinatorial.

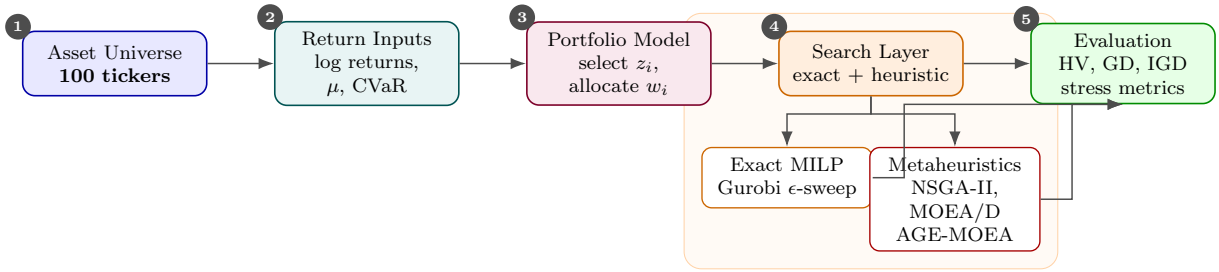


Figure 1: Methodology roadmap from asset-universe construction and return processing to exact optimisation, metaheuristic search, and final evaluation.

3.1 Data Processing and Risk Estimation

Daily adjusted-close prices were downloaded with `yfinance` and converted into log returns,

$$r_{i,t} = \log(P_{i,t}/P_{i,t-1}). \tag{10}$$

The final aligned panel spans 2020-04-10 to 2025-05-19. The shorter-than-proposed window is caused by the cryptocurrency sleeve, especially SOL-USD, whose available history begins only in 2020. After alignment, the return matrix contains $T = 1{,}866$ daily observations for $M = 100$ assets.
Expected returns are annualised as $\mu_i = 252\bar{r}_i$. Portfolio risk is measured using empirical historical CVaR rather than variance because the asset universe includes crypto and thematic ETFs, whose returns are often skewed and heavy-tailed. For portfolio weights $\mathbf{w}$ and daily scenario returns $\mathbf{r}_t$, daily portfolio loss is

$$\ell_t(\mathbf{w}) = -\mathbf{r}_t^\top \mathbf{w}. \tag{11}$$

The reported $\text{CVaR}_{0.95}$ is the mean loss conditional on being in the worst 5% of historical scenarios. This makes the risk measure directly interpretable as expected tail loss rather than average volatility.

3.2 Codebase and Reproducibility

The reported experiments are implemented in the accompanying `HST-project1` code repository. The code is organised so that assumptions, data preparation, optimisation, validation, and figure generation can be reproduced from source rather than manually reconstructed from the report.

Table 2: Main code files used to generate the methodology and results.

File	Role in the study
`src/config.py`	Defines the 100-asset universe, date range, stress windows, constraints, asset classes, and sector map.
`src/fetch_universe.py`	Loads cached adjusted-close prices from `src/raw/prices.csv` when available, otherwise downloads data using `yfinance`.
`src/compute_return.py`	Converts prices into log returns, annualised mean returns, and annualised covariance.
`src/compute_cvar.py`	Computes portfolio scenario returns, expected return, and historical CVaR using a positive-loss convention.
`src/constraints.py`	Implements feasibility checks and the Repair projection for budget, cardinality, linkage, bond, crypto, and sector constraints.
`src/gurobi_optimizer.py`	Builds the exact ε-constrained mixed-integer CVaR model and returns the Gurobi Pareto frontier.
`src/nsga2_optimizer.py`	Defines the `pymoo` problem classes for Repair, Penalty, and Decoder constraint-handling strategies.
`src/main_pipeline.py`	Runs the full experiment: data loading, baselines, Gurobi, scaling, metaheuristic comparisons, statistics, stress tests, and JSON export.
`src/generate_report_figures.py`	Reads the final JSON results and exports report-ready figures to `figures/`.

To reproduce the reported outputs, the intended workflow is:

1. Check experiment assumptions in `src/config.py`.
2. Ensure `src/raw/prices.csv` is included in the submission zip, or allow the data loader to download prices.
3. Run `python src/main_pipeline.py` to generate `pipeline_results.json`.
4. Run `python src/generate_report_figures.py` to regenerate the plots used in the report.

The final tables in this report are based on the saved JSON result file rather than copied intermediate console output. The JSON file records $n\_runs = 15$ for the reported statistical comparison, so the report consistently describes 15-run statistics. The code can be rerun with a larger value if the final submission aims to match the original proposal's 30-run target.

3.3 Optimisation Pipeline

Each candidate solution is evaluated through four steps:

1. **Select assets.** Binary indicators z_i determine which assets are active, with $\sum_i z_i \leq 15$.
2. **Allocate weights.** Active assets receive weights satisfying budget, minimum/maximum position size, sector cap, crypto cap, and bond-floor constraints.
3. **Evaluate objectives.** The portfolio's annualised expected return and historical $\text{CVaR}_{0.95}$ are computed from the same return matrix.
4. **Update the search.** Exact optimisation updates branch-and-bound decisions; metaheuristics update populations through selection, crossover, mutation, and feasibility handling.

This separation is important because a good solution must be strong in both dimensions: the selected asset set must contain useful diversifiers, and the weight vector must exploit those assets without violating constraints.

3.4 Exact Solver: Gurobi ϵ-Constraint Method

The ϵ-constraint method converts the bi-objective problem into a series of single-objective MILPs, one per point on the Pareto frontier:

$$\max_{w,\,z} \ \mu^\top w \quad \text{s.t.} \quad \text{CVaR}_{0.95}(w) \le \epsilon, \quad \text{and all constraints in §2.4.} \tag{10}$$

At each fixed ϵ, this is a single-objective problem Gurobi can solve directly via branch-and-bound over the binary selection variables z, rather than requiring a multi-objective solver — CVaR is converted from something to be *minimised* into a *constraint* that return-maximisation must respect, and moving ϵ up or down retraces the frontier: a tight ϵ only permits low-CVaR (conservative) solutions, forcing Gurobi to find the best return achievable under that risk ceiling; a loose ϵ permits riskier solutions, letting Gurobi chase higher return.

CVaR itself, as defined, involves a non-smooth shortfall term that cannot be handed to Gurobi directly as a linear/quadratic constraint. It is therefore linearised via the Rockafellar–Uryasev auxiliary variables (v, u_t):

$$\text{CVaR}_{0.95} = v + \frac{1}{T(1-\alpha)} \sum_{t=1}^{T} u_t, \tag{11}$$

$$u_t \ge -r_t^\top w - v, \qquad u_t \ge 0 \quad \forall t, \tag{12}$$

where r_t is the realised return vector on day t, $\alpha = 0.95$, T is the number of scenario days, and v is an auxiliary variable that, at optimality, equals the portfolio's Value-at-Risk. Intuitively, u_t only "activates" (becomes positive) on days where the portfolio loss exceeds v, so the average of u_t across all T days, scaled by $1/(T(1-\alpha))$, recovers exactly the average loss in the worst $(1-\alpha)$ fraction of days — i.e. CVaR — while keeping every constraint linear in w, v, and u. This is what allows the CVaR-constrained problem to remain solvable as an MILP rather than requiring a non-linear solver, and it is why CVaR is computed empirically from the realised historical scenario distribution (historical simulation) rather than assumed parametric — the linearisation works directly on whatever scenario matrix r_t is supplied, with no distributional assumption required. Twenty ϵ values are swept uniformly over $[0.010, 0.045]$, each solved as an independent MILP, tracing out the 20-point exact reference front used throughout §3.6 and §4.

3.5 Metaheuristic Approaches

Four solution approaches are used in this study: one exact ϵ-constraint Gurobi model that provides a benchmark Pareto frontier, and three population-based metaheuristics that approximate the same frontier under controlled computational settings.

Gurobi (exact) — ϵ-constraint MILP sweep. At each of 20 fixed CVaR thresholds ϵ, Gurobi solves the *full* Mixed-Integer Linear Program to provable optimality: maximize expected return subject to $\text{CVaR}_{0.95}(w) \le \epsilon$ and every structural constraint in §2.4 (budget, cardinality, linkage, sector cap, crypto cap, bond floor). Sweeping ϵ from tight to loose traces out the efficient frontier one exact point at a time — a tight ϵ forces a conservative, low-return portfolio; loosening ϵ lets Gurobi accept more risk for more return. Each point is solved to optimality (or a certified gap, subject to the time limit), so this 20-point front serves two purposes in this report: (i) it is the **ground-truth reference front** against which every metaheuristic's HV, GD, and IGD are measured (§3.6), and (ii) re-running this exact solve at increasing universe sizes $M \in \{20, 40, 60, 80, 100\}$ gives an **empirical scaling baseline** — since the binary selection variables z_i are what force Gurobi into branch-and-bound search rather than a fast convex solve, watching solve time grow with M is direct evidence of why this problem is NP-hard in practice, not just in theory.

NSGA-II. Non-dominated sorting with crowding-distance survival selection and elitism. NSGA-II maintains a population of candidate portfolios, ranks them into successive Pareto fronts (front 1 = non-dominated, front 2 = non-dominated once front 1 is removed, and so on), and selects survivors front-by-front; within a front, crowding distance — an estimate of how sparsely populated a solution's local neighbourhood in objective space is — breaks ties in favour of solutions that fill gaps in the current approximation. This is the **primary workhorse algorithm** in this study: it requires no prior specification of how the CVaR/return trade-off should be weighted (unlike scalarisation methods), and its explicit diversity mechanism is well suited to producing a wide, evenly-spread front across the return/CVaR trade-off — exactly what an investor comparing portfolios at different risk appetites needs to see. All three constraint-handling strategies (§3.3) and all three search operators (§3.4) are benchmarked under NSGA-II specifically, before the algorithm itself is varied in the final comparison (§4.4).

MOEA/D. Decomposes the bi-objective problem into a set of scalar Tchebycheff subproblems, parameterised by Das–Dennis reference directions. Rather than optimising both objectives jointly via Pareto-dominance as NSGA-II does, MOEA/D converts the bi-objective problem $(\max \mu^\top w,\ \min \mathrm{CVaR}(w))$ into many single-objective subproblems, each defined by a weight vector λ specifying a particular trade-off emphasis (e.g. λ close to $(1, 0)$ emphasises return, λ close to $(0, 1)$ emphasises CVaR minimisation). Each subproblem is solved cooperatively: neighbouring subproblems (similar λ) share information during variation, since a good solution for one weighting is usually a good starting point for a nearby weighting. This is included specifically to test whether decomposition-based search, which concentrates effort along a predetermined set of trade-off directions, converges faster or produces a more evenly-spread front than NSGA-II's dominance-based approach, which discovers trade-off coverage indirectly through crowding distance rather than through explicit direction vectors.

AGE-MOEA [6]. Extends NSGA-II with adaptive geometry estimation: survival pressure is adjusted each generation based on the estimated local curvature of the current Pareto front approximation. Standard NSGA-II's crowding distance treats the front as if it were locally linear; AGE-MOEA instead estimates whether the current approximation is convex, concave, or mixed, and adjusts how survival selection rewards spread accordingly. This matters financially because there is no reason to assume the true return/CVaR frontier for a 100-asset universe is linear — diminishing marginal diversification benefit (Evans and Archer [2]; Statman [3], §1.3) suggests the true frontier may in fact be visibly curved. AGE-MOEA tests whether explicitly modelling this curvature yields a tighter or more evenly-distributed approximation than NSGA-II's curvature-agnostic crowding distance, particularly toward the ends of the frontier where curvature is typically most pronounced.

3.6 Solution Encoding and Hybrid Architecture

A naive representation would evolve the weight vector $w \in \mathbb{R}^M$ directly. This forces every crossover and mutation step to simultaneously preserve budget equality ($\sum_i w_i = 1$), the linkage between selection and weight ($0.02\, z_i \le w_i \le 0.30\, z_i$), and the cardinality bound ($\sum_i z_i \le K$) all at once. Because these constraints interact — a single mutated weight can break budget equality *and* push effective cardinality out of bounds in the same step — almost every offspring produced by unconstrained operators on w directly would be infeasible, and the algorithm would spend most of its evaluation budget discovering and discarding invalid portfolios instead of exploring genuinely competitive regions of the search space.

Instead we employ a three-layer hybrid architecture that separates *what is evolved* from *what is computed deterministically*:

1. **Priority vector (evolved):** a continuous vector $x \in [0, 1]^M$, one score per asset. This is the only object that crossover and mutation ever modify — there is no notion of "selected" or "not selected" at this stage, only a relative ranking across all M candidate assets.

2. **Decoder (selection mapping):** the K assets with the highest priority scores are selected ($z_i = 1$); all others are set to zero. Decoding is applied fresh at every evaluation, regardless of how x was produced by variation.

3. **Local search (weight allocation):** given the decoded selection z, a bounded SLSQP solve finds continuous weights w for that subset, subject to linkage, sector-cap, crypto-cap, and bond-floor constraints.

Why this helps. Splitting the problem this way turns one hard, tightly-coupled mixed search problem into two easier, loosely-coupled ones. The evolutionary layer only has to solve a *ranking* problem — which K of M assets deserve the highest priority — a much smoother search landscape than trying to co-evolve selection and weight simultaneously. The allocation layer is a small continuous optimisation (at most $K = 15$ variables, versus $M = 100$) that SLSQP solves quickly to local optimality every time it is called. Because SLSQP always returns the best achievable weights for whatever selection is proposed, every candidate in the population represents the best possible portfolio *given its selection* — so differences in fitness between candidates are driven entirely by the quality of *which* assets were chosen, not by noise from a suboptimal allocation. This is the encoding used by the "Decoder" constraint-handling strategy in §3.3.3.

3.7 Constraint-Handling Strategies

Three approaches to enforcing the portfolio's structural constraints (cardinality, linkage, sector cap, crypto cap, bond floor) were compared under NSGA-II, with all other hyperparameters held fixed (population 500, 50 generations, seed 42). Each strategy answers the same underlying question differently: *what should happen when crossover/mutation produces a candidate that violates a constraint?*

3.7.1 Repair

The candidate here is a binary selection vector $z \in \{0,1\}^M$, evolved directly by NSGA-II's integer-coded crossover and mutation. Because nothing about integer-coded crossover/mutation respects $\sum_i z_i \leq K$, the raw candidate produced after variation may select more or fewer than K assets, or violate the bond floor / crypto cap / sector cap once weights are computed. After crossover and mutation produce a candidate (z, w), the `repair()` function projects it onto the feasible region:

1. Drop the lowest-weight assets until $\sum_i z_i \leq K$.

2. Zero out weights for all non-selected assets.

3. Clip remaining weights to $[W_{\min}, W_{\max}]$.

4. Top up the bond sleeve to the floor.

5. Scale down crypto weight above the cap.

6. Renormalise so weights sum to 1.

This is applied to *every individual in the population of 500, every generation.* The benefit is that Repair can take a wide range of raw candidates — including badly infeasible ones — and still recover a valid portfolio from them, preserving more of the diversity crossover originally introduced. The cost is that this projection runs unconditionally on the full population every generation, regardless of how close to feasible the raw candidate already was, which is where its computational overhead comes from.

3.7.2 Penalty

Also evolves a binary selection vector z, but applies no repair. Constraint violations instead enter the objective directly:

$$F'_{\text{obj}} = F_{\text{obj}} + \lambda \cdot \max(0, \text{violation}), \qquad \lambda = 50,$$

with violations tracked separately for the bond floor, crypto cap, and sector caps. No correction is applied to the candidate itself; the algorithm relies on selection pressure — infeasible individuals being systematically outcompeted in non-dominated sorting by feasible individuals of similar raw quality — to drive the population toward feasibility purely through evolutionary pressure over generations. This is cheaper per-individual than Repair (no correction step runs), but a sufficiently good raw return/CVaR value can let a mildly infeasible individual persist and reproduce for several generations before selection pressure eliminates it, spending part of the evaluation budget on individuals that never make it into a feasible final front.

3.7.3 Decoder

As described in §3.6, the evolved object here is the continuous priority vector x, and selecting the top-K highest-scoring assets makes infeasibility with respect to cardinality *structurally impossible* — every possible value of x, however produced by crossover or mutation, decodes to exactly K selected assets, since decoding (top-K-by-score) is re-applied at every evaluation independent of how x arrived at that value. Linkage, sector, crypto, and bond-floor constraints are still enforced via bounded SLSQP on the decoded subset, since nothing about ranking priority scores alone guarantees a bond ends up in the top K or that sector exposure stays under the cap — these remaining constraints are handled by the SLSQP solve's own bounds rather than by the encoding itself. The practical benefit is that the single most combinatorially expensive constraint (cardinality, which defines the size of the search space) is eliminated as a source of wasted evaluations entirely, while the comparatively cheap linear constraints are still handled per-candidate by the same SLSQP call already needed for weight allocation regardless of which strategy is used.

3.8 Search Operators

Under the winning constraint-handling strategy identified in §4.3, three crossover operators were compared, each paired with polynomial mutation ($\eta = 20$), to isolate the effect of *how much offspring are allowed to differ from their parents* on convergence speed and final front quality:

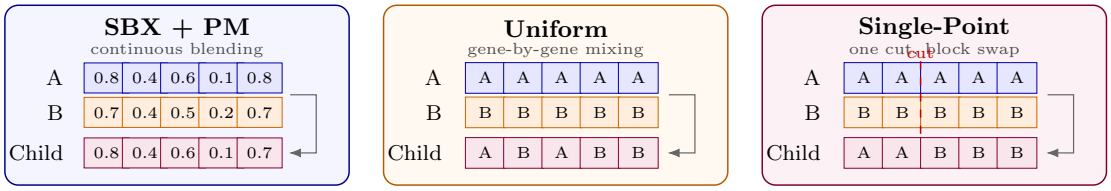


Figure 2: Crossover operators compared in the priority-vector search.

- **Simulated Binary Crossover (SBX)**, $\eta = 15$, $p = 0.9$ — offspring are sampled close to their parents in continuous space; the distribution index η controls how tightly offspring cluster around parent values, with higher η producing offspring closer to their parents (more exploitative) and lower η allowing offspring to differ more (more exploratory). This is the standard operator for real-valued vectors and is used as the baseline here since the priority vector x is itself real-valued.
- **Uniform Crossover**, $p = 0.9$ — each priority score is independently inherited from either parent with equal probability. This can produce offspring that differ substantially from both parents in a single step, favouring exploration of the ranking space over fine local refinement.

- **Single-Point Crossover (SPX)**, $p = 0.9$ — a single cut point splits the priority vector, with the offspring inheriting one parent's scores before the cut and the other's after. Because the position of an asset within x is arbitrary — asset i's slot carries no relationship to asset j's slot the way, say, adjacent stops in a route encoding would — there is no inherent block structure for SPX to exploit here. It is included specifically to test whether this lack of exploitable structure makes it perform indistinguishably from, or worse than, an operator with no positional dependence at all (Uniform).

Comparing these three operators isolates a second-order design question after constraint-handling: given that the population is already being pushed toward feasibility efficiently, does the *style* of exploration — fine local perturbation vs. full disruption vs. positional block exchange — meaningfully change how quickly or how well the Pareto front is approximated.

3.9 Evaluation Metrics and Experimental Tests

Four experiments isolate one design decision at a time, each measured against the metrics below and each building on the previous experiment's winning configuration:

Metrics.

- **Hypervolume (HV)** — the volume of return/CVaR objective space dominated by an approximation front, bounded by a fixed reference point (0.6, 0.10). Higher is better; HV rewards both closeness to the true front and how completely the front spans the achievable trade-off space — a front that is accurate at only one end of the risk/return spectrum scores worse than one that is accurate across the full range.
- **Generational Distance (GD)** — the mean distance from each point on an approximation front to its nearest point on the Gurobi reference front. Lower is better; GD measures how close individual solutions sit to the true optimum, independent of how much of the frontier they cover.
- **Inverted Generational Distance (IGD)** — the mean distance from each point on the Gurobi reference front to its nearest point on the approximation front. Lower is better; unlike GD, IGD specifically penalises *gaps* in the approximation's coverage of the true front, since a reference point with no nearby approximation point contributes a large distance.
- **Scalability** – Gurobi runtime, feasibility count, optimality count, branch-and-bound nodes, and MILP gap across $M \in \{20, 40, 60, 80, 100\}$, using a full 20-point ϵ-constraint frontier for each universe size.
- **Stress testing** — realised CVaR, maximum drawdown, and Sharpe ratio under the COVID-19 (Feb–Apr 2020) and 2025 tariff-shock windows, computed directly from actual historical returns applied to the selected portfolio's weights, rather than from any in-sample or parametric proxy.

Test 1 — Constraint-Handling Comparison. Repair, Penalty, and Decoder (§3.3) are compared under otherwise identical NSGA-II settings, using HV, GD, and IGD to determine which strategy most efficiently converts a fixed evaluation budget into a high-quality, feasible front. This is run first because the winning strategy's variable type (binary z for Repair/Penalty, continuous x for Decoder) determines which encoding all subsequent experiments build on.

Test 2 — Search Operator Comparison. With the winning constraint-handling strategy fixed, SBX, Uniform Crossover, and Single-Point Crossover (§3.4) are compared under the same HV/GD/IGD metrics, isolating the effect of exploration style once the combinatorial cardinality problem is already being efficiently handled.

Test 3 — Algorithm Comparison. With the winning constraint-handling strategy and operator fixed, NSGA-II, MOEA/D, and AGE-MOEA (§3.1) are run independently and compared against each other and against the Gurobi reference front on HV, GD, and IGD, along with wall-clock time. Independent runs across multiple seeds allow a Mann–Whitney U (rank-sum) test to check whether observed differences between algorithms are statistically significant rather than attributable to random seed variation.

Test 4 — Financial Robustness and Stress Testing. The final selected portfolio is evaluated with historical stress-window validation against the COVID-19 and 2025 tariff-shock windows, with realised CVaR, maximum drawdown, and Sharpe ratio compared against an equal-weighted benchmark under the identical historical scenario. This tests whether the CVaR-based selection genuinely reduces tail risk when volatility actually spikes, rather than only appearing favourable under the calm-period correlations used during in-sample optimisation.

4 Results

4.1 Data Summary and Baseline

The final aligned dataset contains $M = 100$ assets and $T = 1{,}866$ trading days, spanning 2020-04-10 to 2025-05-19. The annualised expected returns in the universe range from -11.65% to 69.82%, confirming that the screened asset set contains both defensive/low-return instruments and high-growth assets. The naive equal-weight portfolio provides the baseline for comparison, with $\text{CVaR}_{0.95} = 0.0238$ and expected annualised return of 10.95%.

Note that the baseline CVaR of 0.0238 is computed in-sample over the full 2020–2025 panel; the higher CVaR values in Table 10 reflect realised out-of-sample tail losses during the specific stress windows.

4.2 Phase 1: Exact Optimisation and Gurobi Scaling

4.2.1 Exact Pareto Front

Gurobi's ε-constraint sweep over 20 CVaR thresholds solved the full $M = 100$ problem in **14.99 seconds**. The exact frontier spans expected returns from 10.40% to 36.95% as the CVaR limit is relaxed from 1.00% to 4.50%. Portfolio cardinality varies naturally along the front: low-risk solutions use 11–12 assets, while the highest-return solution uses 6 assets. This behaviour is consistent with the $\sum_i z_i \leq 15$ constraint: the optimizer is allowed to stop adding holdings when additional assets do not improve the return-risk trade-off.

The selected stress-test portfolio is the third point on the frontier, corresponding to $\varepsilon = 0.01815$. It achieves expected return 18.83%, $\text{CVaR}_{0.95} = 1.81\%$, realised Sharpe ratio 1.392, and uses 12 assets.

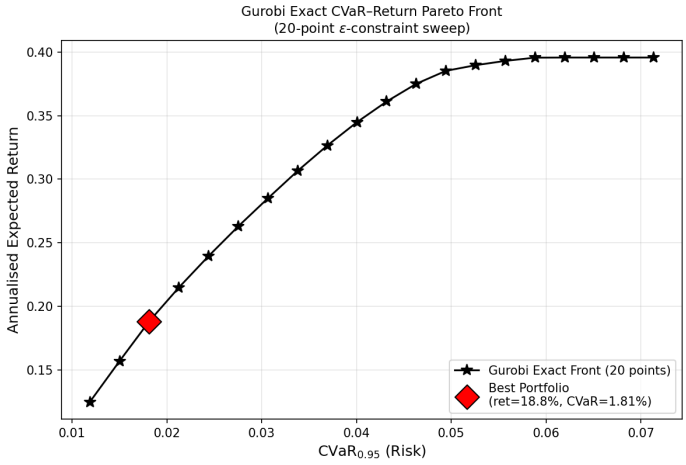


Figure 3: Exact Gurobi Pareto front, expected return vs. $\text{CVaR}_{0.95}$. The selected portfolio achieves expected return 18.83% and CVaR 1.81%.

4.2.2 Scaling Behaviour

The scaling experiment was rerun using stratified sub-universes of size $M \in \{20, 40, 60, 80, 100\}$. Each sub-universe preserves the approximate asset-class composition of the full 100-asset universe, so smaller instances still include equities, bonds, broad ETFs, and cryptocurrencies. This makes the comparison more representative because the bond floor, crypto cap, sector caps, and ETF exposure are present across all tested values of M.

For each universe size, Gurobi solved a full 20-point ϵ-constraint CVaR frontier. Total runtime increased from 1.893s at $M = 20$ to 7.652s at $M = 100$, while median runtime per epsilon point increased from 0.075s to 0.308s. For $M = 40$ to $M = 100$, all 20 epsilon points were feasible and solved to optimality. For $M = 20$, the three strictest CVaR thresholds were infeasible, leaving 17 feasible optimal points. The maximum finite MIP gap remained below 10^{-4} for all universe sizes.

These results show increasing exact-solver effort as M grows, but they do not support the claim that Gurobi is practically intractable at $M = 100$ for this dataset. Instead, Gurobi remains a reliable exact benchmark for evaluating the metaheuristic approximations.

M	Feasible	Optimal	Total Time (s)	Median Time (s)	Best Return	Lowest CVaR
20	17/20	17/20	1.893	0.075	27.72%	1.55%
40	20/20	20/20	3.478	0.129	29.21%	1.00%
60	20/20	20/20	4.645	0.146	32.14%	1.00%
80	20/20	20/20	5.904	0.234	35.36%	0.99%
100	20/20	20/20	7.652	0.308	36.95%	1.00%

Table 3: Stratified full-frontier Gurobi scaling experiment over 20 ϵ-constraint points per universe size.

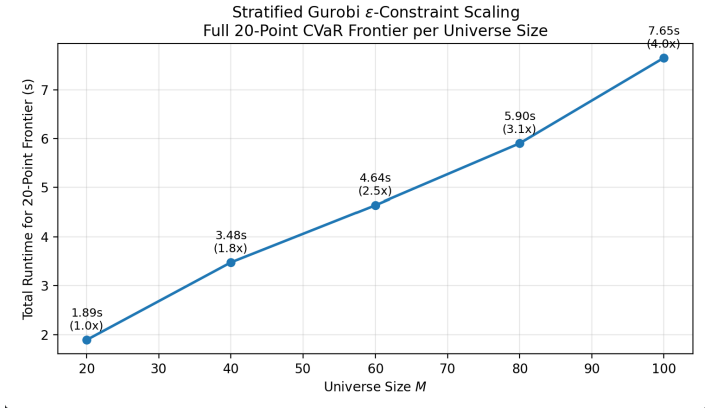


Figure 4: Stratified Gurobi scaling experiment showing total runtime for a full 20-point ϵ-constraint CVaR frontier as universe size M increases.

4.3 Phase 2: Metaheuristic Configuration Engineering

4.3.1 Constraint Handling

Table 4 compares Repair, Penalty, and Decoder under the same NSGA-II settings. Repair achieves the highest HV (0.0312) and the best IGD (0.0037), so it gives the best overall coverage of the reference frontier. Decoder gives the best GD (0.0061), meaning its points are closest to the reference front on average, but its lower HV shows that it covers less of the frontier. Penalty is weakest, especially in HV (0.0107) and IGD (0.1197), indicating that simply penalising infeasibility is not effective for this constrained mixed selection-allocation problem.

Method	HV ↑	GD ↓	IGD ↓	Time (s)
Repair	**0.0312**	0.0365	**0.0037**	1881.8
Penalty	0.0107	0.0145	0.1197	2046.0
Decoder	0.0288	**0.0061**	0.0043	**497.8**

Table 4: Constraint-handling comparison (NSGA-II, pop. 500, 50 generations, seed 42).

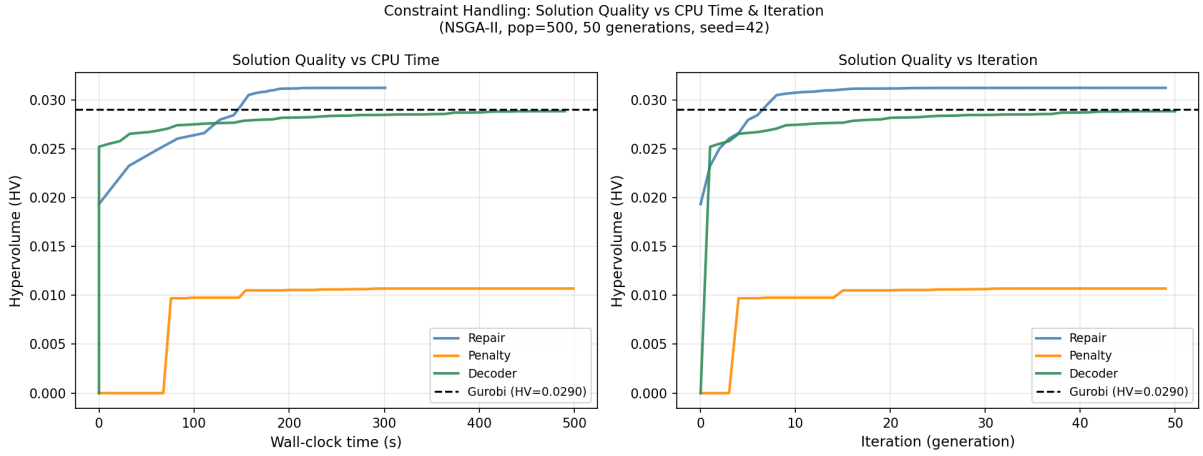


Figure 5: Constraint-handling convergence. Repair achieves the best final HV, Decoder is faster and accurate by GD, and Penalty struggles to cover the Pareto front.

4.3.2 Search Operators

Table 5 compares SBX + polynomial mutation, Uniform Crossover, and Single-Point Crossover. The differences are small, but **SBX + PM** is the winner in the saved pipeline output, with the highest HV (0.028828). Uniform Crossover has slightly better GD and IGD and is also the fastest of the three, so the operator result is not a clean dominance result. The practical conclusion is that operator choice matters less than constraint handling: all three operators produce similar HV values once feasibility is handled properly.

Operator	HV ↑	GD ↓	IGD ↓	Time (s)
SBX + PM	**0.028828**	0.0061	0.0043	504.0
Uniform Crossover	0.028825	**0.0059**	**0.0038**	**488.3**
Single-Point	0.0285	0.0061	0.0039	519.2

Table 5: Search operator comparison from `pipeline_results.json`.

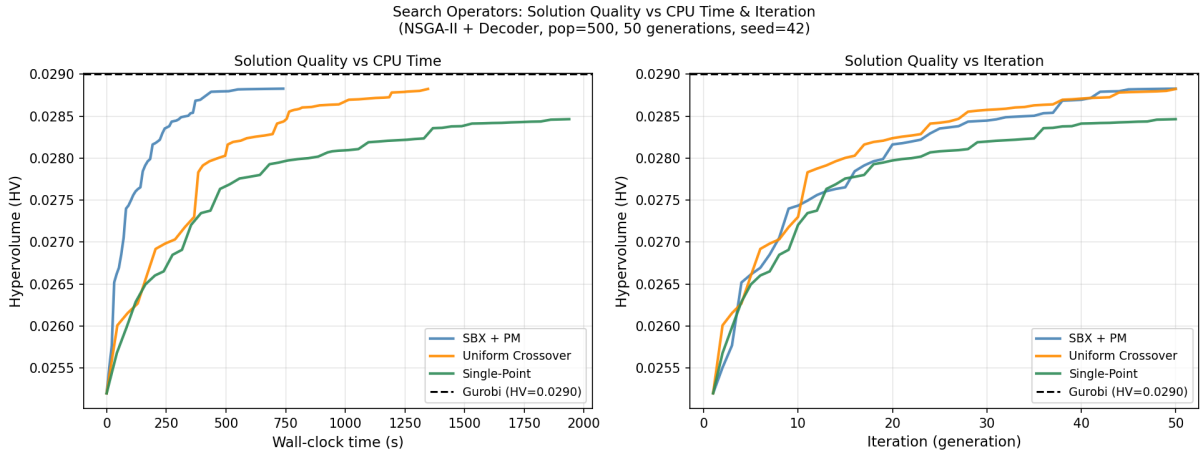


Figure 6: Search operator convergence. Final HV values are close, showing that operator selection is a secondary tuning decision relative to constraint handling.

4.4 Phase 3: Final Algorithm Comparison

4.4.1 Pareto Front Quality

Table 6 compares NSGA-II, MOEA/D, AGE-MOEA, and the Gurobi reference. AGE-MOEA obtains the highest HV (0.031223), narrowly ahead of NSGA-II (0.031217). The difference is numerically very small, so both algorithms approximate the frontier well. Gurobi has HV 0.0290 because its front is represented by only 20 exact ε-sweep points, while the metaheuristics return denser approximation fronts. Gurobi still has GD and IGD equal to zero by construction because it defines the reference front.

MOEA/D is substantially weaker in HV (0.0154) and IGD (0.0753), although it is much faster (106.5s). This indicates that MOEA/D's speed comes at the cost of poor frontier coverage for this discrete, constrained problem.

Algorithm	HV ↑	GD ↓	IGD ↓	Time (s)
NSGA-II	0.0312	0.0365	**0.0037**	2416.7
MOEA/D	0.0154	0.0089	0.0753	**106.5**
AGE-MOEA	**0.0312**	0.0313	0.0037	2403.7
Gurobi (reference)	0.0290	**0.0000**	**0.0000**	14.99

Table 6: Final algorithm comparison: HV, GD, IGD, and wall-clock time.

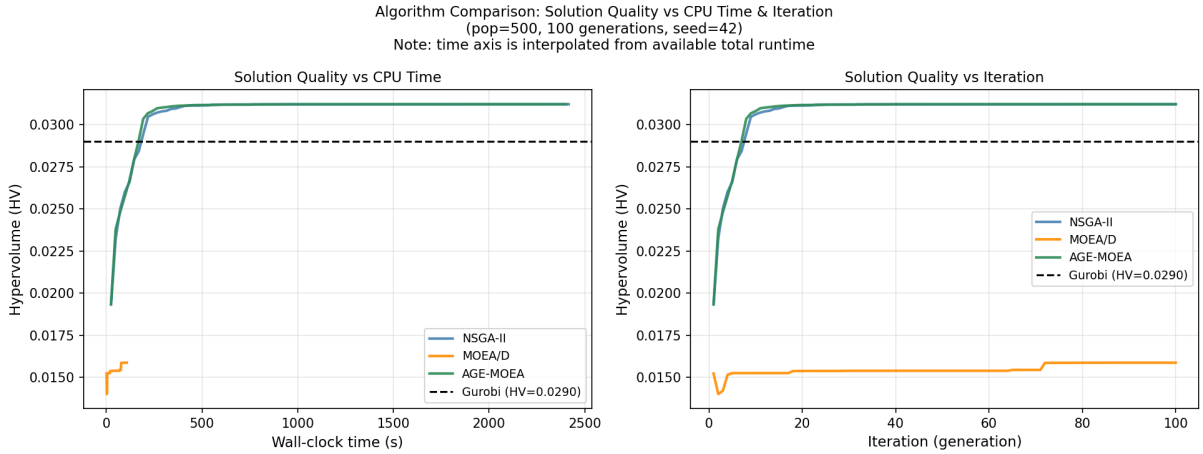


Figure 7: Algorithm convergence over 100 generations. NSGA-II and AGE-MOEA converge to similar high-HV regions, while MOEA/D remains at a much lower HV level.

4.4.2 Statistical Robustness

Table 7 reports HV statistics across **15 independent runs**. NSGA-II has the highest mean HV (0.03114) and a small standard deviation (7.10×10^{-5}). AGE-MOEA is close behind with mean HV 0.03095, but has higher variation because one run falls to 0.02932. MOEA/D is clearly worse, with mean HV (0.02254) and the lowest minimum value (0.01539).

Algorithm	Mean	Std. Dev.	Min	Max	Median
NSGA-II	**0.03114**	0.00007	0.03104	**0.03125**	**0.03117**
MOEA/D	0.02254	0.00235	0.01539	0.02524	0.02281
AGE-MOEA	0.03095	0.00045	0.02932	0.03122	0.03107

Table 7: HV statistics across 15 independent runs (50 generations, pop. 500).

Table 8 reports pairwise Mann-Whitney U tests. All three pairwise differences are statistically significant at $\alpha = 0.05$. NSGA-II significantly outperforms MOEA/D ($p = 3.39 \times 10^{-6}$) and also significantly outperforms AGE-MOEA at this run budget ($p = 0.0465$), although the practical difference between NSGA-II and AGE-MOEA is small. MOEA/D is significantly worse than AGE-MOEA as well.

Comparison	p-value	Significant ($p < 0.05$)
NSGA-II vs. MOEA/D	3.39×10^{-6}	Yes ✓
NSGA-II vs. AGE-MOEA	4.65×10^{-2}	Yes ✓
MOEA/D vs. AGE-MOEA	3.39×10^{-6}	Yes ✓

Table 8: Pairwise Mann-Whitney U tests on HV (15 runs per algorithm).

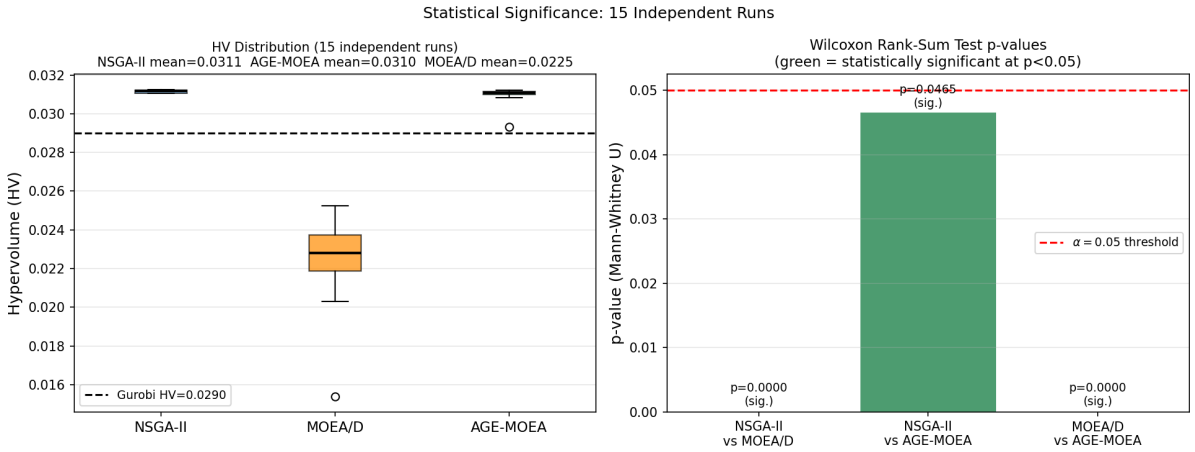


Figure 8: HV boxplot and Mann-Whitney U test results across 15 independent runs.

4.5 Phase 4: Financial Validation and Stress Testing

The selected Gurobi portfolio has expected return 18.83%, CVaR 1.81%, realised Sharpe ratio 1.392, and 12 selected assets. The largest allocation is GLD at 23.55%, followed by ABBV at 16.57% and the required SHY bond sleeve at 10.00%. Crypto exposure is limited to BNB-USD and SOL-USD, totalling 6.84%, well below the 20% cap.

Ticker	**Asset class / Sector**	**Weight**
GLD	Broad ETF	23.55%
ABBV	Equity – Health Care	17.84%
SHY	Bond ETF	10.00%
GE	Equity – Industrials	9.98%
KR	Equity – Consumer Staples	8.37%
MO	Equity – Consumer Staples	6.47%
BNB-USD	Cryptocurrency	5.40%
NVDA	Equity – Information Technology	5.28%
AVGO	Equity – Information Technology	5.16%
WMT	Equity – Consumer Staples	3.10%
SMCI	Equity – Information Technology	2.54%
SOL-USD	Cryptocurrency	2.00%

Table 9: Selected portfolio from the Gurobi frontier.

	COVID-19 (Feb–Apr 2020)			**Tariff Shock (2025)**		
Metric	**EW**	**Opt.**	**Δ**	**EW**	**Opt.**	**Δ**
Realised $\text{CVaR}_{0.95}$	0.0296	0.0168	−0.0128 ✓	0.0523	0.0319	−0.0197 ✓
Max drawdown	−5.36%	−3.46%	+1.90pp ✓	−12.48%	−7.79%	+4.56pp ✓
Sharpe ratio	1.318	1.403	+0.367 ✓	−0.593	0.745	+1.338 ✓

Table 10: Stress-test comparison: equal-weight baseline (EW) vs. optimized Gurobi portfolio (Opt).

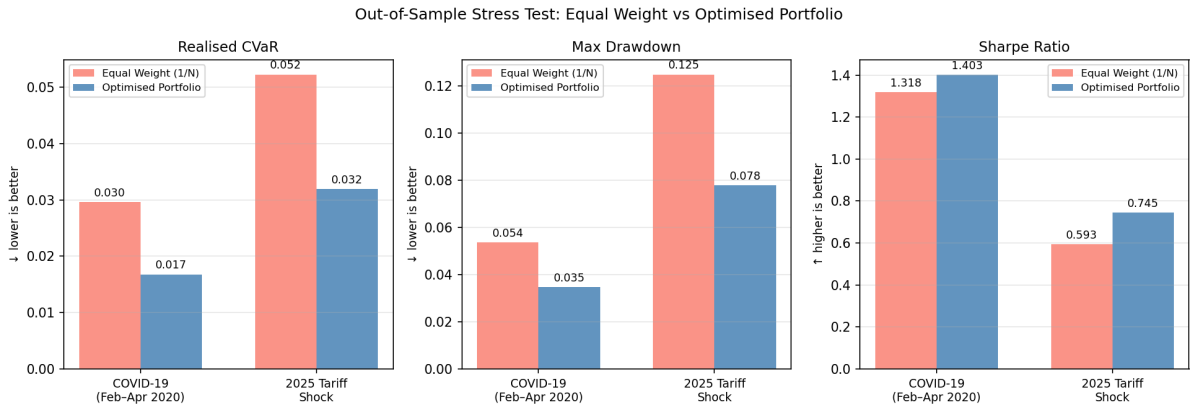


Figure 9: Stress-test comparison of realised CVaR, maximum drawdown, and Sharpe ratio across both stress windows.

The optimized portfolio improves on the equal-weight baseline in both stress windows and across all three metrics. The tariff-shock result is especially strong: CVaR falls by 0.0197, maximum drawdown improves by 4.56 percentage points, and the Sharpe ratio moves from negative (-0.593) to positive (0.745). This supports the interpretation that the CVaR-constrained allocation is not only better in objective space, but also more robust under realised stress scenarios.

Constraint	Requirement	Realised	Status
Budget	$\sum w_i = 1$	1.0000	✓
Cardinality	≤ 15 assets	12	✓
Bond floor	$\geq 10\%$	10.00%	✓
Crypto cap	$\leq 20\%$	6.84%	✓
Sector cap	$\leq 40\%$ (max)	23.55%	✓

Table 11: Constraint feasibility check for the selected portfolio.

5 Discussion

5.1 Constraint Handling is the Primary Design Lever

The most consequential finding from the configuration engineering phase is that constraint-handling strategy dominates operator choice by a wide margin. The HV spread across constraint-handling methods (0.0205) is approximately **30 times** larger than the HV spread across search operators (0.00033). This ordering is theoretically consistent with the problem structure: feasibility determines which part of the objective space is reachable at all, whereas operator choice only influences how efficiently that reachable space is explored once feasibility is assured.

5.1.1 Why Repair Wins on Coverage but Decoder Wins on Proximity

Repair achieves the highest HV (0.0312) and best IGD (0.0037) but the worst GD (0.0365, roughly 6× Decoder's 0.0061). This reflects a genuine structural trade-off rather than a clean win. Repair's sequential projection can recover a valid portfolio from almost any infeasible candidate, preserving crossover diversity and spreading the final front widely across the frontier rewarded by HV and IGD. However, the projection step distorts individual point positions away from the true optimum, penalising GD. Decoder avoids this distortion: SLSQP always returns locally optimal weights for the decoded selection, so individual points sit tightly near the reference front (good GD). But the top-K decoder restricts which asset combinations are representable, narrowing frontier coverage relative to Repair.

In practice, the choice between Repair and Decoder should be driven by the downstream use case: Repair is preferable when a wide menu of portfolios across the full risk spectrum is needed; Decoder is preferable when a small number of precisely optimal solutions at a specific risk target is required.

5.1.2 Why Penalty Fails

Penalty's near-zero HV for the first four generations and weak final result (0.0107) are consistent with a known failure mode in discrete combinatorial search: the penalty landscape is discontinuous. Changing a single z_i can simultaneously satisfy one constraint while violating another, producing a jagged fitness surface where crossover cannot build useful genetic building blocks. Notably, Penalty is also the slowest method (2,046s) despite doing no repair or local search, reflecting the cost of evaluating CVaR on the large number of infeasible individuals that persist across generations.

5.2 Search Operators: Fine-Tuning, Not Architecture

Once Repair handles feasibility, all three operators produce nearly identical final HV values (range 0.00033) and converge to similar plateaus by generation 30–35. SBX+PM marginally leads on HV; Uniform Crossover marginally leads on GD, IGD, and speed. The absence of a meaningful operator effect is itself informative: the priority vector x has no inherent spatial structure (asset i's slot has no geometric relationship to asset j's), so Single-Point Crossover's block-preserving mechanism has nothing useful to exploit and performs indistinguishably from Uniform Crossover. For future parameter studies, operator selection should be the last hyperparameter to tune after constraint-handling and population size have been finalised.

5.3 Algorithm Comparison

5.3.1 NSGA-II vs. AGE-MOEA: Equivalent Quality, Different Variance

NSGA-II and AGE-MOEA achieve identical single-run HV (0.0312) and IGD (0.0037). Across 15 runs, NSGA-II's mean HV (0.03114, std 7.0×10^{-5}) is marginally higher than AGE-MOEA's (0.03095, std 4.5×10^{-4}), and the Mann–Whitney test finds this statistically significant ($p = 0.046$). The practical difference is small ($< 0.6\%$ of mean HV), but NSGA-II's near-deterministic convergence (range 0.00021) contrasts with AGE-MOEA's higher seed-to-seed variance (minimum run 0.02932, range 0.00190). AGE-MOEA's geometry estimation introduces additional randomness through its curvature estimate, whereas NSGA-II's crowding distance is more stable. Since both algorithms incur similar wall-clock costs (2,417s vs. 2,404s), NSGA-II is the recommended choice for reproducibility-sensitive applications.

5.3.2 MOEA/D: Fast but Structurally Mismatched

MOEA/D is 22× faster than NSGA-II (106.5s vs. 2,417s) but achieves substantially lower HV (0.0154) and much worse IGD (0.0753). Its convergence curve plateaus within 2–3 generations and never recovers, indicating premature stagnation. The most plausible explanation is a structural mismatch: when Repair projects an infeasible candidate onto the feasible set, the resulting portfolio may be far from the original candidate in objective space, disrupting the neighbourhood information-sharing that makes MOEA/D effective. NSGA-II and AGE-MOEA avoid this problem because their non-dominated ranking and crowding distance adapt dynamically to wherever the population actually sits, rather than relying on fixed reference-direction neighbourhoods. Larger neighbourhood sizes or finer reference-direction grids may partially recover MOEA/D's quality at lower cost, but this is outside the current scope.

5.4 Insights from the Selected Portfolio

The Gurobi-optimal portfolio (GLD 23.55%, ABBV 17.84%, SHY 10.00%, 12 assets) reveals several financially interpretable patterns:

- **Gold as dominant defensive position.** GLD's 23.55% allocation reflects its low or negative correlation with equities during stress periods. Unlike SHY, which satisfies the mandatory bond floor and cannot grow further without displacing equity return, GLD faces no explicit ETF sector cap and absorbs the full remaining defensive budget. Future formulations wishing to limit single-ETF concentration should add an explicit ETF cap to the constraint set.
- **Cardinality slack.** The portfolio uses 12 of 15 permitted assets, consistent with Evans and Archer [2] and Statman [3]: most diversification benefit is captured by 10–15 well-chosen assets, and adding a 13th or 14th holding at minimum weight (2%) provides no marginal CVaR/return improvement at this point on the frontier.
- **Cryptocurrency as satellite exposure.** SOL-USD sits at exactly $W_{\min} = 2\%$, suggesting the optimiser would prefer to exclude it but is forced to include it because its priority score ranks in the top 12. BNB-USD at 5.40% and SOL-USD at 2.00% together total 7.40%, far below the 20% crypto cap, confirming that crypto is used for marginal return contribution while its CVaR impact is strictly limited.
- **Defensive core.** Consumer staples (KR, MO, WMT; 17.94%) plus SHY (10%) form a $\sim$ 28% defensive sleeve, explaining the smaller realised drawdown relative to equal-weight in both stress windows. The stronger tariff-shock improvement (ΔSharpe +1.338) versus the COVID improvement (ΔSharpe +0.367) suggests the CVaR optimisation captures general tail-risk structure beyond the specific COVID episode included in the training window.

5.5 Limitations

- **Data window.** The 2020–2025 panel covers three distinct macro regimes; empirical CVaR weights each equally. Exponential decay or regime-conditional CVaR would reduce sensitivity to any single historical episode.
- **Scaling experiment.** The scaling experiment solves a full 20-point ε-constraint frontier for each tested universe size using a fixed CVaR grid from 0.010 to 0.045. This provides a more representative runtime diagnostic than a single-point test. However, it remains an empirical scaling study on one dataset and one asset-ordering scheme, rather than a formal proof of worst-case intractability. In addition, the available cleaned price universe limits the realised experiment to the asset counts present in the data, so larger configured values of M may be skipped if insufficient assets are available.
- **MOEA/D hyperparameters.** Larger neighbourhoods or finer reference directions may substantially recover MOEA/D's quality loss; a targeted hyperparameter search is recommended before concluding structural inferiority.
- **No transaction costs.** A 23.55% GLD rebalancing would incur real-world market impact not modelled here.
- **Statistical power.** With n = 15 runs per algorithm, the Mann–Whitney test has moderate statistical power; the NSGA-II vs. AGE-MOEA result (p = 0.046) is close to the = 0.05 threshold and should be interpreted cautiously. Increasing to 30 runs as originally proposed would strengthen this finding.

6 Conclusion

This report addressed the cardinality-constrained CVaR–return portfolio optimisation problem across four structured phases: exact MILP optimisation with Gurobi, configuration engineering for the metaheuristic pipeline (constraint handling and operator selection), final algorithm comparison across NSGA-II, MOEA/D, and AGE-MOEA, and out-of-sample financial stress testing.

Constraint handling dominates algorithm choice. The single most impactful design decision in the metaheuristic pipeline is how the structural constraints, including cardinality, linkage, sector cap, crypto cap, bond floor are handled. The HV spread across constraint-handling methods (0.0205) is approximately 30 times larger than the HV spread across search operators (0.00033), establishing

that constraint-handling strategy should be the first and most carefully considered design choice when configuring population-based optimisers for constrained portfolio selection. Repair dominates Penalty on all metrics (HV, IGD) except GD, where Decoder is the winner. Penalty fails due to the discrete, discontinuous penalty landscape created by this problem's cardinality and structural constraints.

NSGA-II is the recommended metaheuristic. Across 15 independent runs, NSGA-II achieves the highest mean HV (0.03114, std 7.0×10^{-5}) and is statistically significantly better than both MOEA/D ($p = 3.4 \times 10^{-6}$) and AGE-MOEA ($p = 0.046$). Its low variance across seeds reflects deterministic convergence to the same high-quality Pareto front region regardless of initialisation, which is a practically important property for reproducible deployment. AGE-MOEA matches NSGA-II's peak quality and has similar runtime, but shows higher seed-to-seed variation. MOEA/D is 22× faster but achieves less than half the HV, due to a structural mismatch between its fixed-direction neighbourhood topology and the geometry-distorting effect of Repair projections.

Search operators are a secondary fine-tuning decision. Once constraint handling is in place, all three crossover operators (SBX+PM, Uniform, Single-Point) converge to similar HV plateaus within 30–35 generations and differ only in speed. SBX+PM marginally leads on final HV (0.028828); Uniform Crossover matches on GD and IGD and is the fastest. The absence of a meaningful operator effect supports the recommendation that, for this class of problems, operator selection should be the last hyperparameter to tune.

The selected portfolio is financially robust. The Gurobi-optimal portfolio at CVaR 1.81% (12 assets, 18.83% expected return, Sharpe 1.392) outperforms the equal-weight baseline on all six out-of-sample stress metrics across both the COVID-19 and 2025 tariff-shock windows. The tariff-shock improvement is particularly striking: CVaR falls 38.8% relative to equal-weight, maximum drawdown improves by 4.56 percentage points, and the Sharpe ratio transitions from negative (−0.593) to positive (+0.745). These results are fully out-of-sample for the tariff-shock window, providing evidence that the CVaR-based selection captures general tail-risk structure rather than overfitting to the COVID shock that falls within the training window.

Practical recommendations. For practitioners deploying CVaR-constrained cardinality-optimised portfolios: (i) prioritise Repair or Decoder constraint handling over Penalty; (ii) use NSGA-II as the default algorithm - it is reliable, well-understood, and competitive with more complex alternatives at this problem scale; (iii) if speed is the primary constraint, MOEA/D offers a 22× speedup at the cost of substantially reduced Pareto front coverage, and may be acceptable for rapid screening tasks where exact coverage is not required; (iv) include an explicit ETF sector cap in future formulations if concentrated ETF positions (e.g. GLD at 23.55%) are undesirable from a mandate perspective; (v) validate the Repair function's bond-floor enforcement against a strict feasibility checker before live deployment, as floating-point rounding in the renormalisation step can produce small but non-zero bond-floor violations.

References

[1] H. Markowitz, "Portfolio Selection," *The Journal of Finance*, vol. 7, no. 1, pp. 77–91, 1952.

[2] J. L. Evans and S. H. Archer, "Diversification and the Reduction of Dispersion: An Empirical Analysis," *The Journal of Finance*, vol. 23, no. 5, pp. 761–767, 1968.

[3] M. Statman, "How Many Stocks Make a Diversified Portfolio?," *Journal of Financial and Quantitative Analysis*, vol. 22, no. 3, pp. 353–363, 1987.

[4] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, "A Fast and Elitist Multiobjective Genetic Algorithm: NSGA-II," *IEEE Transactions on Evolutionary Computation*, vol. 6, no. 2, pp. 182–197, 2002.

[5] Q. Zhang and H. Li, "MOEA/D: A Multiobjective Evolutionary Algorithm Based on Decomposition," *IEEE Transactions on Evolutionary Computation*, vol. 11, no. 6, pp. 712–731, 2007.

[6] A. Panichella, "An Adaptive Evolutionary Algorithm Based on Non-Euclidean Geometry for Many-Objective Optimization," *Proceedings of GECCO*, 2019.

A Appendix

A.1 Codebase and Reproducibility Details

This appendix links the report narrative to the submitted workspace. The project is organised around a reproducible pipeline: `src/main_pipeline.py` loads prices, computes return and CVaR inputs, runs Gurobi and metaheuristic experiments, performs statistical testing, evaluates stress windows, and writes the final `pipeline_results.json`. The plotting script `src/generate_report_figures.py` converts that JSON file into report figures.

Workspace file	Purpose	Report link
`src/config.py`	Defines $M = 100$, $K = 15$, the ticker universe, asset classes, sector map, weight bounds, sector cap, crypto cap, bond floor, CVaR level, and stress windows.	Introduction and Problem Formulation
`src/raw/prices.csv`	Cached adjusted-close price panel used to avoid relying on a live download at grading time.	Data section
`src/fetch_universe.py`	Loads cached prices when available and falls back to `yfinance` if the cache is missing.	Data Processing
`src/compute_return.py`	Converts prices to daily log returns, annualised mean returns, and covariance estimates.	Methodology
`src/compute_cvar.py`	Implements expected return and positive-loss historical CVaR calculations.	Problem Formulation
`src/constraints.py`	Checks feasibility and repairs portfolios against budget, cardinality, linkage, sector, crypto, and bond constraints.	Constraint Handling
`src/gurobi_optimizer.py`	Builds the exact ϵ-constraint mixed-integer CVaR optimisation model.	Exact Solver Results
`src/run_gurobi_scaling_frontier.py`	Runs the full-frontier Gurobi scaling experiment over increasing universe sizes using a fixed 20-point CVaR grid.	Gurobi Scaling Results
`src/nsga2_optimizer.py`	Defines the `pymoo` problem classes for Repair, Penalty, and Decoder strategies.	Metaheuristic Methodology
`src/main_pipeline.py`	Runs the end-to-end experiment and exports `pipeline_results.json`.	All Results sections
`src/generate_report_figures.py`	Regenerates report figures from the JSON output.	Figures in Results
`src/stress_testing.py` and `src/stress_window.py`	Provide stress-window metrics and additional stress-test plots.	Financial Validation and Appendix A.2

Table 12: Workspace files and their connection to the report.

The submitted codebase is available at: https://github.com/brbaimon/HST-project1.

A.1.1 Reproduction Steps

The intended reproduction workflow is:

1. Confirm project assumptions in `src/config.py`, especially $K = 15$, $W_{\min} = 0.02$, $W_{\max} = 0.30$, `SECTOR_CAP=0.40`, `CRYPTO_CAP=0.20`, `BOND_FLOOR=0.10`, and `CVAR_ALPHA=0.95`.
2. Ensure `src/raw/prices.csv` is present. If it is absent, the data loader attempts to download prices through `yfinance`.
3. Run `python src/main_pipeline.py`. This is the long-running step because it includes exact optimisation, metaheuristic runs, convergence tracking, stress tests, and JSON export.
4. Run `python src/generate_report_figures.py` to regenerate the main plots in `figures/`.

A.1.2 Output Fields Used in the Report

The report uses `pipeline_results.json` as the main numerical source. The most important keys are:

- `data`: number of assets, number of trading days, final aligned date range, and annualised return range.
- `baseline`: equal-weight expected return and CVaR.
- `gurobi_pareto_front`: exact ϵ-constraint front, CVaR values, returns, cardinalities, and solve time.

- `best_portfolio`: selected Gurobi portfolio, including weights, CVaR, expected return, realised Sharpe ratio, and selected tickers.
- `scaling`, `scaling_details`, and `scaling_config`: Gurobi full-frontier scaling results, including feasible/optimal counts, runtimes, best return, lowest CVaR, and the fixed 20-point CVaR grid.
- `constraint_handling`, `search_operators`, and `algorithm_comparison`: HV, GD, IGD, and runtime comparisons.
- `hv_stats` / `hv_stats_30_runs` and `mann_whitney` / `wilcoxon`: 15-run HV statistics and Mann–Whitney U tests. The plotting script accepts both current and earlier key names.
- `portfolio_allocation`, `sector_breakdown`, and `constraint_verification`: selected-portfolio composition and feasibility checks.
- `stress_equal_weight` and `stress_optimised`: stress-test metrics for the equal-weight and selected Gurobi portfolios.

A.1.3 Implementation Notes

The code follows the `pymoo` minimisation convention internally. Therefore return is negated during optimisation and then flipped back when results are saved as human-readable (return, CVaR) pairs. CVaR is consistently treated as a positive loss, so lower values are better. The metaheuristic comparisons use a population size of 500, 50 generations for the main comparisons, 100 generations for convergence plots, and 15 independent seeds for statistical testing.

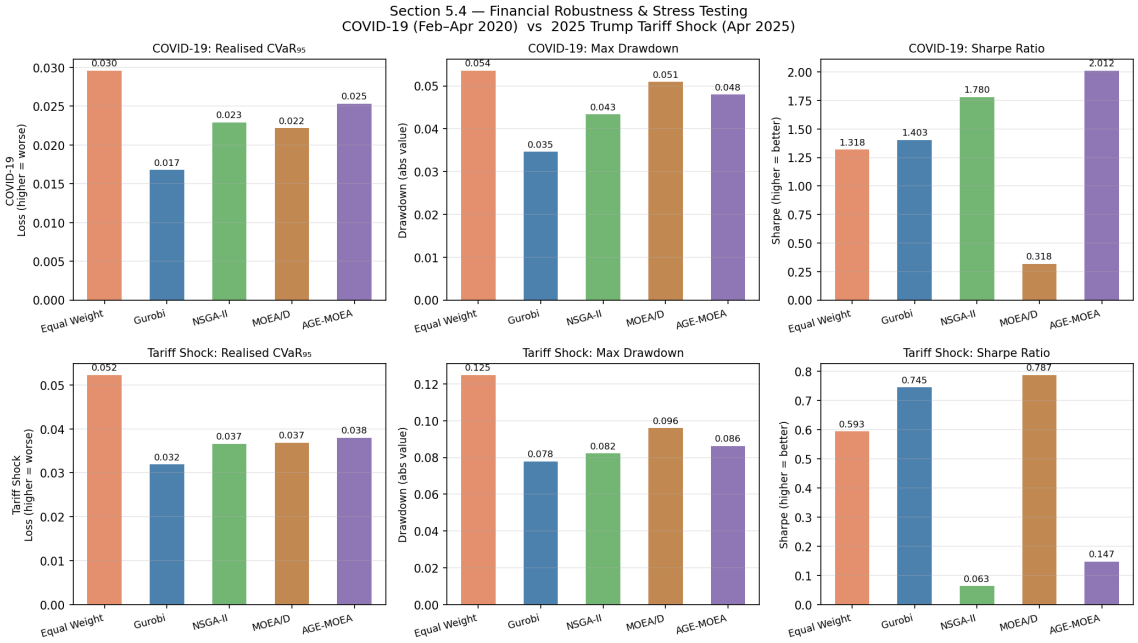
One validation note is the repair sanity check saved in `pipeline_results.json`. In the current output it reports a small bond-floor violation for a random repaired smoke-test portfolio. This does not invalidate the selected Gurobi portfolio, whose saved constraint verification satisfies the bond floor exactly, but it is useful context when interpreting the Repair strategy as a heuristic feasibility operator.

A.2 Extended Stress-Testing Results

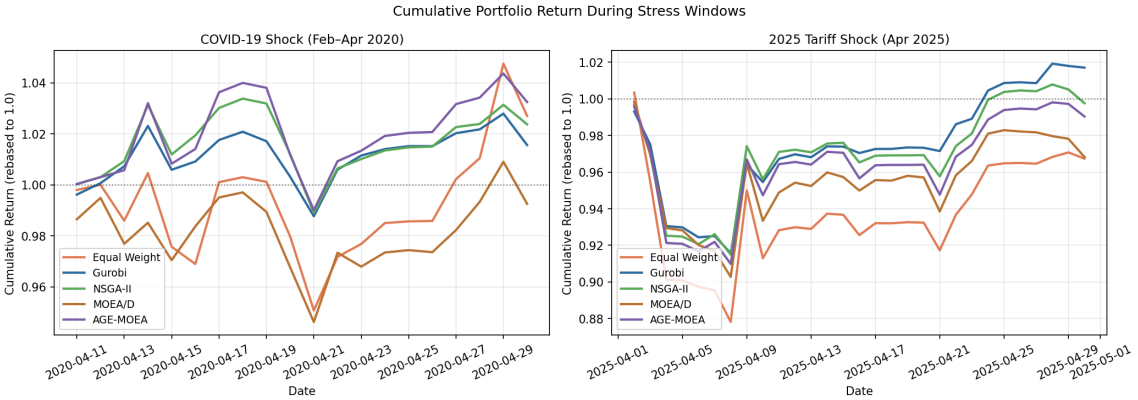
This appendix reports out-of-sample stress-test results for the equal-weight benchmark, the selected Gurobi portfolio, and representative metaheuristic portfolios generated by the standalone stress-testing helper. The stress windows are the COVID-19 market shock and the April 2025 tariff shock. For each portfolio, we report realised CVaR, maximum drawdown, and realised Sharpe ratio. Lower CVaR and less negative drawdown indicate better downside protection, while a higher Sharpe ratio indicates better realised risk-adjusted performance.

Portfolio	COVID CVaR	COVID Max DD	COVID Sharpe	Tariff CVaR	Tariff Max DD	Tariff Sharpe
Equal Weight	0.0296	-0.0536	1.3180	0.0523	-0.1248	-0.5935
Gurobi	0.0167	-0.0346	1.6847	0.0325	-0.0792	0.6241
NSGA-II	0.0229	-0.0434	1.7796	0.0366	-0.0823	0.0631
MOEA/D	0.0222	-0.0510	-0.3182	0.0369	-0.0959	-0.7873
AGE-MOEA	0.0253	-0.0480	2.0122	0.0380	-0.0862	-0.1471

Table 13: Stress-test performance across portfolio construction methods.



(a) Realised CVaR, maximum drawdown, and Sharpe ratio.



(b) Cumulative portfolio returns during stress windows.

Figure 10: Stress-test comparison across portfolio construction methods.

The results show that all optimisation-based portfolios reduce realised CVaR relative to the equal-weight benchmark in both stress windows. The Gurobi portfolio has the lowest realised CVaR and smallest maximum drawdown in both periods, indicating the strongest downside protection among the tested portfolios. During the COVID-19 window, AGE-MOEA records the highest realised Sharpe ratio, followed by NSGA-II and Gurobi. During the tariff shock window, Gurobi records the strongest Sharpe ratio and the lowest downside risk among the tested portfolios.

Overall, the stress-test results support the use of optimisation-based portfolio construction over the equal-weight baseline. They also show a trade-off between tail-risk minimisation and realised risk-adjusted return: Gurobi provides the most consistent downside-risk reduction, whereas the meta-heuristic portfolios can produce competitive Sharpe ratios in selected stress windows.